



Resolución automática de problemas utilizando búsqueda



Métodos de búsqueda no informados ("ciegos")

- ✓ búsqueda en profundidad (DFS)
- ✓ búsqueda a la ancho (BFS)
- ✓ Low cost-first (LCFS)

Métodos de búsqueda informados ("heurísticos")

- El mejor primero (Best-FS)
- en profundidad heurístico (HDFS)
- A*
- Hill Climbing (HC)
- Stochastic HC (recocido simulado)
- Local Beam Search



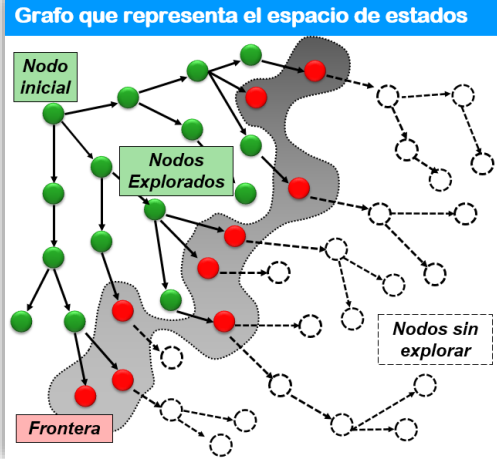
Algoritmo de búsqueda genérico



Buscar una meta en la frontera F del grafo de estado

Caso base:
cuando **elijo** un nodo N de F y N satisface el test de meta

Caso recursivo:
cuando **elijo** un nodo N de F y N no es una meta, entonces, primero sacar a N de F, luego **agregar** a la frontera todos los nodos a los que puedo llegar con algún operador aplicable a N (podría no haber ninguno), y finalmente **buscar una meta en la nueva frontera**.

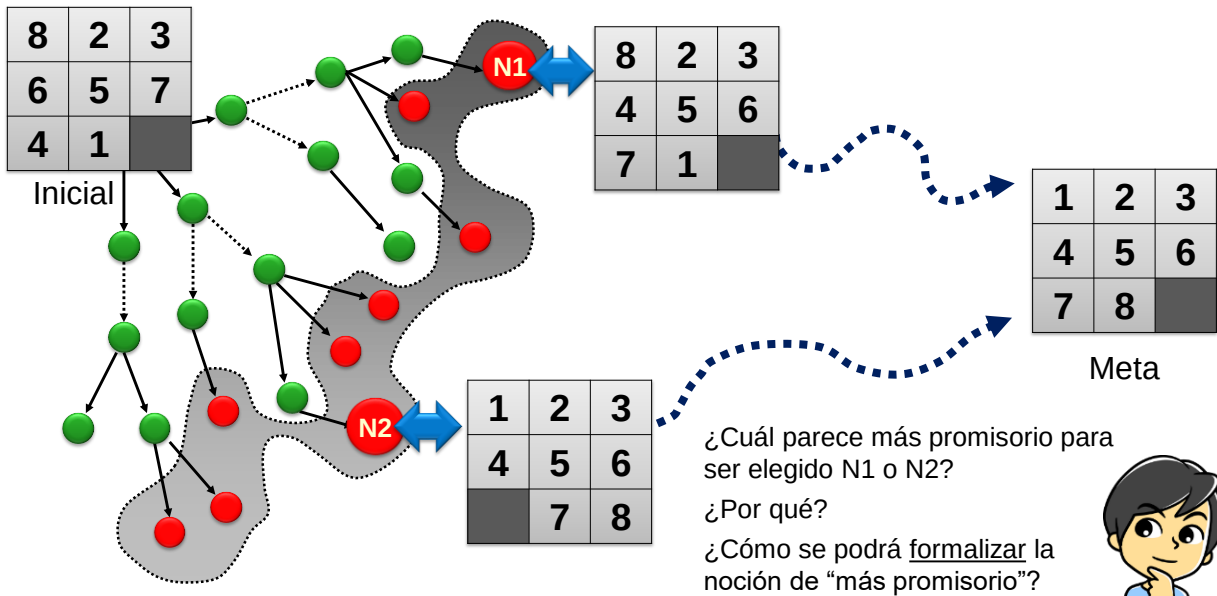


Elijo	Agregar	Estrategia
El primero de "Frontera"	Al principio de Frontera	búsqueda en profundidad
El primero de "Frontera"	Al final de Frontera	búsqueda a lo ancho

IA - Alejandro J. García



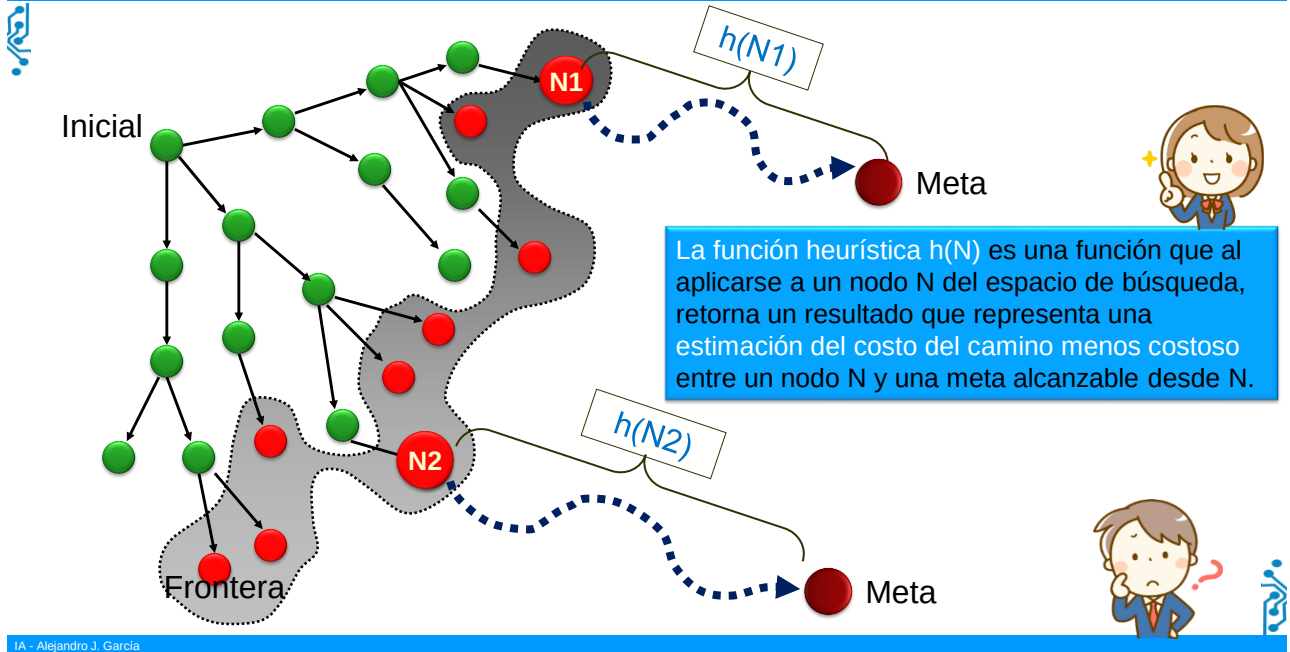
¿Por qué no elegir el nodo más promisorio de la frontera?



IA - Alejandro J. García



Función heurística



Función heurística: conceptos

- En los métodos de búsqueda informada se **utiliza la información presente en los estados** para realizar una **estimación** de cual nodo en la frontera pueda ser **más promisorio** para llegar más rápidamente a una meta.
- Esta información llamada “**heurística**” es **particular al dominio de aplicación** en el que se realiza la búsqueda.
- Para un nodo N , se representa esa **información heurística**, con una función llamada $h(N)$ que **retorna un valor no negativo**.
- El valor de $h(N)$ es una **estimación del costo del camino menos costoso** entre N y una meta alcanzable desde N .
- En general, cada problema tiene su propia $h(N)$ y para cada problema se pueden definir varias $h(N)$.



Más sobre el término heurística en: <http://en.wikipedia.org/wiki/Heuristic>
y en: <http://lema.rae.es/drae/?val=heurística>

Ejemplo: función heurística para 8-tejas

El resultado de una función heurística $h(N)$ es un valor no negativo que representa una estimación del costo del camino menos costoso entre un nodo N y una meta alcanzable desde N .

La función heurística $h(N)$ valora la información que puede ser obtenida rápidamente de un nodo N .

N1:

1	2	3
4	8	5
	7	6

N2:

1	2	3
4	5	6
	7	8

N3:

8	2	3
4	5	6
7	1	

N4:

8	2	3
6	5	7
4	1	

En general, cada problema tiene su propia $h(N)$ y para cada problema se pueden definir varias $h(N)$.

Ejemplo: $h_d(N)$ = cantidad de fichas desacomodadas en N

- $h_d(N1) = 4$
- $h_d(N2) = 2$
- $h_d(N3) = 2$
- $h_d(N4) = 5$

Claramente
es una
estimación

Meta:

1	2	3
4	5	6
7	8	

IA - Alejandro J. García

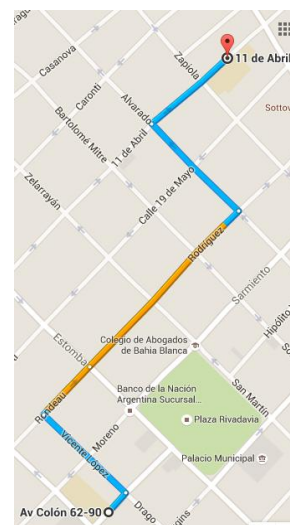
Ejemplos de funciones heurísticas para 8-tejas

El valor de $h(N)$ es una estimación del costo del camino menos costoso entre N y una meta alcanzable desde N .

	<table><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>6</td></tr><tr><td></td><td>7</td><td>8</td></tr></table>	1	2	3	4	5	6		7	8	
1	2	3									
4	5	6									
	7	8									
N2:											

	<table><tr><td>8</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>6</td></tr><tr><td>7</td><td>1</td><td></td></tr></table>	8	2	3	4	5	6	7	1		
8	2	3									
4	5	6									
7	1										
N3:											

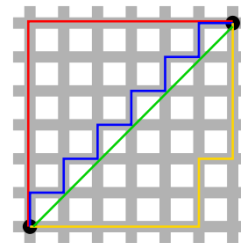
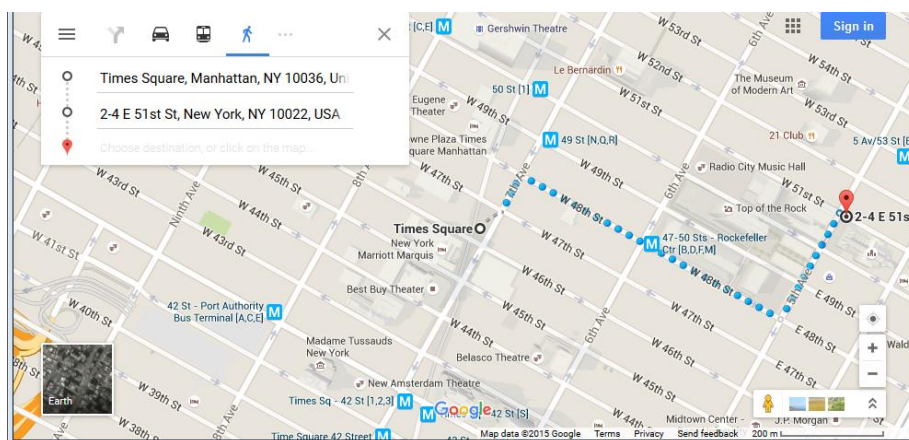
- La cantidad de fichas fuera de lugar es una estimación, ya que el costo real depende de los movimientos necesarios para acomodar cada ficha.
- Esta distancia entre dos puntos del tablero se conoce generalmente con el nombre de distancia de Manhattan, porque es análogo al cálculo de cuantas cuadras hay que caminar para llegar de un punto 'A' a otro 'B' en una ciudad con manzanas regulares como en la ciudad de Manhattan o Bahía Blanca ☺.



IA - Alejandro J. García

“Distancia de Manhattan” Geometría Taxicab

En la Geometría Taxicab, (Hermann Minkowski, siglo XIX), la métrica usual de la geometría euclídeana es reemplazada por una nueva métrica en la cual la distancia entre dos puntos es la suma de las diferencias (absolutas) de sus coordenadas. La métrica Taxicab también se conoce como distancia rectilínea, distancia de ciudad, o longitud Manhattan. El último nombre alude al diseño de grilla de la mayoría de las calles de la isla de Manhattan. https://en.wikipedia.org/wiki/Taxicab_geometry



IA - Alejandro J. García

Ejemplos de funciones heurísticas para 8-tejas

El valor de $h(N)$ es una estimación del costo del camino menos costoso entre N y una meta alcanzable desde N .



Comparación de dos funciones heurísticas:

$h_d(N)$ = cantidad de fichas desacomodadas en N

- $h_d(N1) = 4$
- $h_d(N2) = 2$
- $h_d(N3) = 2$
- $h_d(N4) = 5$

$h_s(N)$ = suma de las “distancias (taxicab)” de cada ficha a su lugar

- $h_s(N1) = 1 + 1 + 1 + 1 = 4$
- $h_s(N2) = 1 + 1 = 2$
- $h_s(N3) = 3 + 3 = 6$
- $h_s(N4) = 3 + 1 + 2 + 3 + 3 = 12$

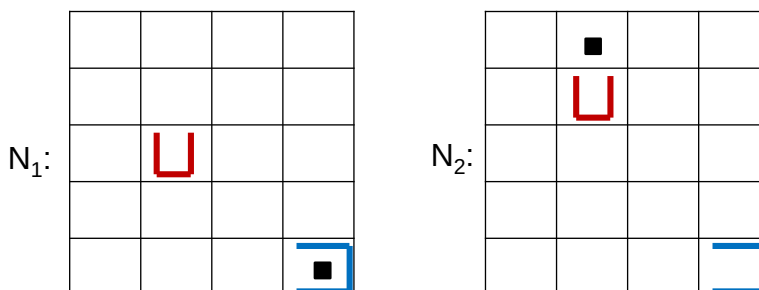
Meta:

1	2	3
4	5	6
7	8	

IA - Alejandro J. García

Función heurística para otro problema

El valor de $h(N)$ es una estimación del costo del camino menos costoso entre N y una meta alcanzable desde N .



En general, cada problema tiene su propia $h(N)$ y para cada problema se pueden definir varias $h(N)$.

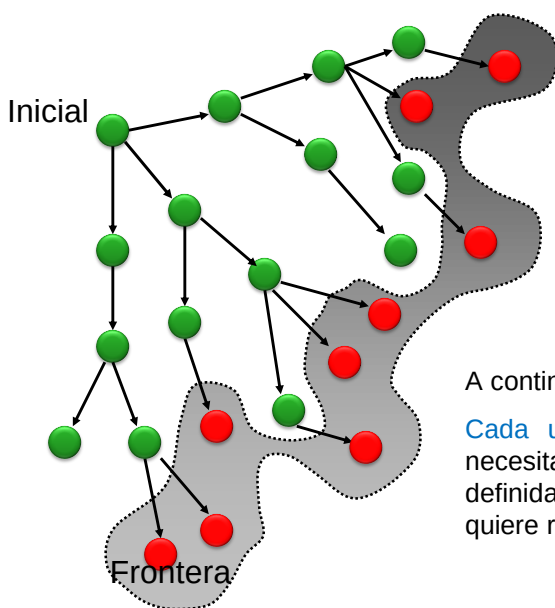
Una posible heurística: $h_c(N) = \text{"distancia" del cursor a caja roja} + \text{"distancia" de la caja roja a la caja azul}$.

$$h_c(N_1) = 4+4=8$$

$$h_c(N_2) = 1+5=6$$

IA - Alejandro J. García

Búsqueda informada



En los métodos de búsqueda informada se utiliza una función heurística aplicada a la información presente en los estados para realizar una estimación de cual nodo en la frontera pueda ser más promisorio para llegar más rápidamente a una meta.

A continuación veremos métodos de búsqueda informada.

Cada uno utiliza una estrategia diferente pero todos necesitan de una función heurística que deberá estar definida considerando los detalles del problema que se quiere resolver.

IA - Alejandro J. García

Método de búsqueda informada

El mejor primero (Best-First Search)

El Mejor Primero (Best-First Search o BestFS)



Estrategia: elegir de la frontera el nodo que tenga menor valor de $h(N)$, esto es, el que “parece estar más cerca de la meta”.

 $h(i)=2$

- La frontera se trata como una cola con prioridad, ordenada por $h(N)$.
- Para cada nodo N se muestra el valor de $h(N)$.



 $m1 \quad h(m1)=0$

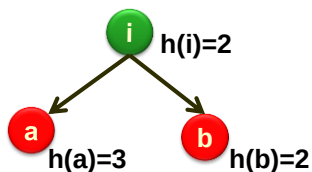
 $m2 \quad h(m2)=0$



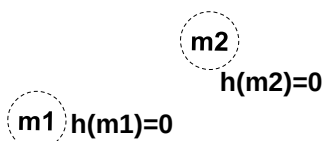
El Mejor Primero (Best-First Search o BestFS)



Estrategia: elegir de la frontera el nodo que tenga menor valor de $h(N)$, esto es, el que “parece estar más cerca de la meta”.



- La frontera se trata como una cola con prioridad, ordenada por $h(N)$.
- Para cada nodo N se muestra el valor de $h(N)$.



$h(N)$ es una estimación del costo del camino menos costoso entre un nodo N y una meta alcanzable desde N

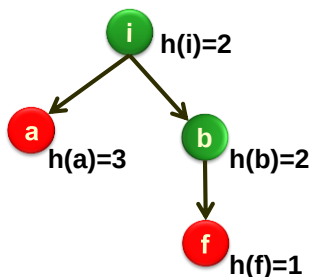


IA - Alejandro J. García

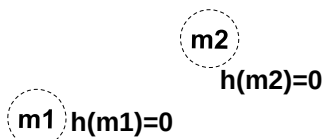
El Mejor Primero (Best-First Search o BestFS)



Estrategia: elegir de la frontera el nodo que tenga menor valor de $h(N)$, esto es, el que “parece estar más cerca de la meta”.



- La frontera se trata como una cola con prioridad, ordenada por $h(N)$.
- Para cada nodo N se muestra el valor de $h(N)$.
- En este ejemplo, y con esta heurística **BestFS** encuentra la meta $m2$.



$h(N)$ es una estimación del costo del camino menos costoso entre un nodo N y una meta alcanzable desde N

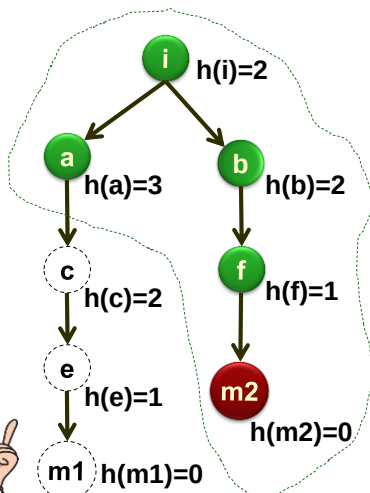


IA - Alejandro J. García

El Mejor Primero (Best-First Search o BestFS)



Estrategia: elegir de la frontera el nodo que tenga menor valor de $h(N)$, esto es, el que “parece estar más cerca de la meta”.



IA - Alejandro J. García

- La frontera se trata como una cola con prioridad, ordenada por $h(N)$.
- Para cada nodo N se muestra el valor de $h(N)$.
- En este ejemplo, y con esta heurística **BestFS** encuentra la meta $m2$.
- El espacio explorado es únicamente el que está dentro de la línea punteada.
- La complejidad espacial puede ser un grave problema ya que en el peor caso es exponencial.

$h(N)$ es una estimación del costo del camino menos costoso entre un nodo N y una meta alcanzable desde N



El Mejor Primero (Greedy Best-First Search)



Estrategia: elegir de la frontera el nodo que tenga menor valor de $h(N)$, esto es, el que “parece estar más cerca de la meta”.

Seleccionar	Agregar	Estrategia
El primero	Ordenar frontera por $h(N)$	Mejor primero (BestFS)

Se puede implementar así:

seleccionar(Nodo, [Nodo | RestoLista] , RestoLista).

agregar(ViejaF, Vecinos, NuevaF) :-
 append(Vecinos, ViejaF, Aux),
 ordenar_por_h(Aux, NuevaF).

Observe que la frontera se trata como una **cola con prioridad**, ordenada por $h(N)$.

$h(N)$ es una estimación del costo del camino menos costoso entre un nodo N y una meta alcanzable desde N



IA - Alejandro J. García

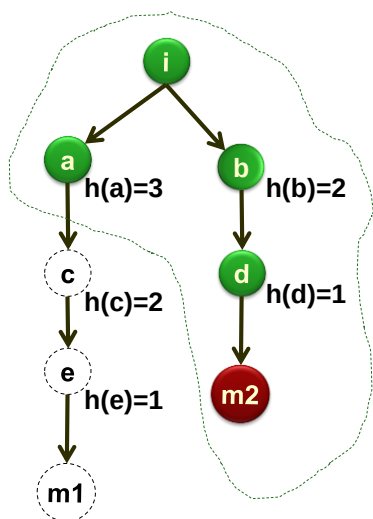
Método de búsqueda informada:

Profundidad heurístico (Heuristic Depth-First Search)

Heuristic Depth-First Search (HDFS)



Estrategia (profundidad heurístico): elije de los primeros vecinos del nodo elegido el de menor valor de $h(N)$.



- El espacio explorado es únicamente el que está dentro de la línea punteada.
- En este ejemplo, y con esta heurística ambos métodos (HDFS y BestFS) encuentran la meta m2.

Tareas importantes:

- Mostrá un ejemplo donde BestFS y HDFS encuentren distintas metas.
- ¿Por qué HDFS mejora la complejidad espacial?



Heuristic Depth-First Search (HDFS)



Estrategia (profundidad heurística): elije de los sucesores del último nodo elegido el de menor valor de $h(N)$.

Seleccionar	Agregar	Estrategia
El primero	Ordenar frontera por $h(N)$	Mejor primero (BestFS)
El primero	Al principio pero ordenados por $h(N)$	Profund. heurístico (HDFS)

seleccionar(Nodo, [Nodo | RestoLista] , RestoLista).

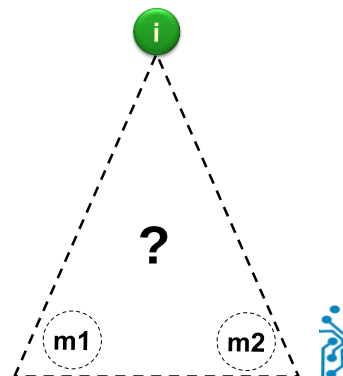
agregar(ViejaF, Vecinos, NuevaF) :-
 ordenar_por_h(Vecinos, Ordenados),
 append(Ordenados, ViejaF, NuevaF).

Esta estrategia tiene un mejor manejo de memoria que BestFS.

La función $h(N)$ se usa para ordenar los vecinos del nodo elegido último, antes de agregar al principio de la frontera.



Tarea: Mostrá un ejemplo donde BestFS y HDFS encuentren distintas metas.



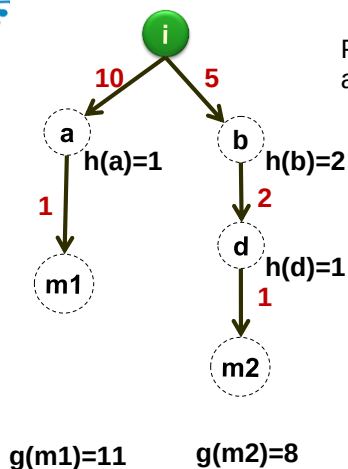
IA - Alejandro J. García

Ejemplo con información heurística



Considerá este espacio de estados con arcos de costo diferente.

Para cada nodo N se muestra a un costado el valor de $h(N)$ y en el arco el costo de ese paso.



Terminología y Conceptos Básicos

Un **problema de búsqueda** se define indicando:

1. La descripción del **estado inicial** desde donde comenzar la búsqueda.
2. La descripción de los **operadores disponibles** con los cuales el agente puede transformar un estado en otro.
3. Un **test de meta** que determina si un estado es una meta.
4. Una **función de costo de camino $g(n)$** que determina el costo para llegar del estado inicial a otro estado " n ".

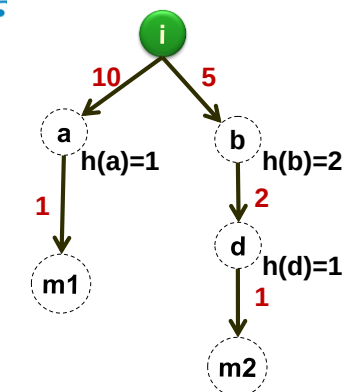
Una **solución** a un problema de búsqueda **es un camino desde el estado inicial a una meta** (puede no ser única).

Una **solución es óptima** si tiene **menor costo** de camino que todas las demás soluciones.



IA - Alejandro J. García

Ejemplo con información heurística



$g(m1)=11$

$g(m2)=8$

Realizá la búsqueda con los métodos:

“el mejor primero” y “profundidad heurística”

¿Qué meta encuentran como solución estos dos métodos?

¿Por qué te parece que pasa esto?

¿Qué es lo que dirige la búsqueda?



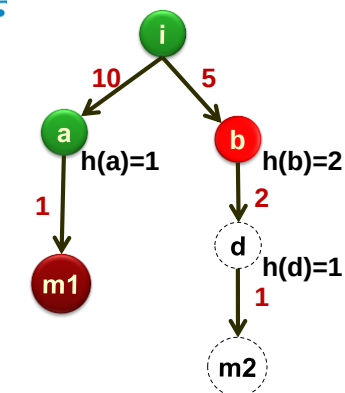
Una solución a un problema de búsqueda es un camino desde el estado inicial a una meta (puede no ser única).

Una solución es óptima si tiene menor costo de camino que todas las demás soluciones.



IA - Alejandro J. García

Ejemplo con información heurística



$g(m1)=11$

$g(m2)=8$

Realizá la búsqueda con los métodos:

“el mejor primero” y “profundidad heurística”

¿Qué meta encuentran como solución estos dos métodos?

¿Por qué te parece que pasa esto?

¿Qué es lo que dirige la búsqueda?



Una solución a un problema de búsqueda es un camino desde el estado inicial a una meta (puede no ser única).

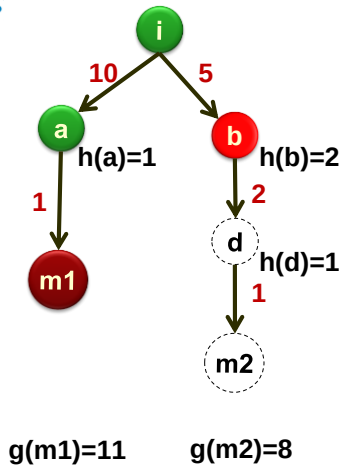
Una solución es óptima si tiene menor costo de camino que todas las demás soluciones.



IA - Alejandro J. García

Ejemplo con información heurística

La funciones $h(N)$ y $g(N)$ representan **conceptos diferentes**.



Terminología y Conceptos Básicos

Un **problema de búsqueda** se define indicando:

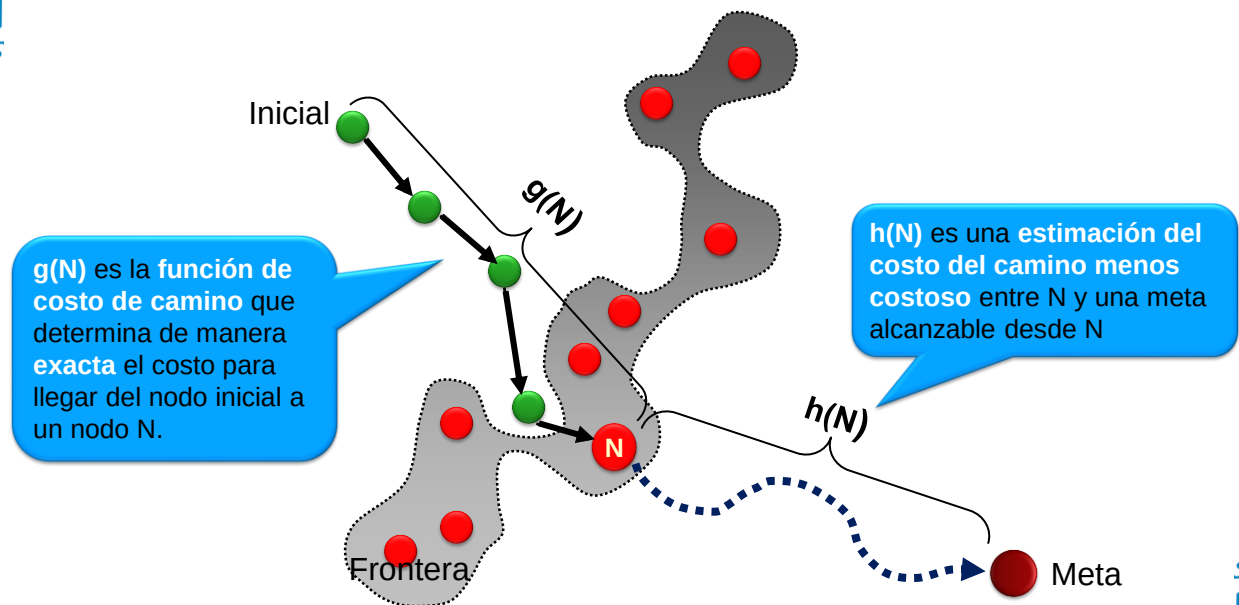
1. La descripción del **estado inicial** desde donde comenzar la búsqueda.
2. La descripción de los **operadores disponibles** con los cuales el agente puede transformar un estado en otro.
3. Un **test de meta** que determina si un estado es una meta.
4. Una **función de costo de camino $g(n)$** que determina el costo para llegar del estado inicial a otro estado "n".

Una **solución** a un problema de búsqueda es un camino desde el estado inicial a una meta (puede no ser única).

Una **solución es óptima** si tiene **menor costo** de camino que todas las demás soluciones.

IA - Alejandro J. García

Considerando "el pasado y el futuro"



IA - Alejandro J. García

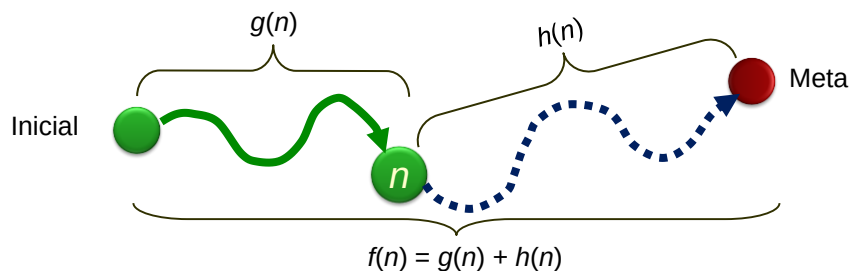
Método de búsqueda informada

A^* (A estrella)

Método de búsqueda A^*



Estrategia de A^* : elegir de la frontera el nodo n con menor $f(n) = g(n) + h(n)$



Hart P. E, Nilsson H. J. and Raphael B. (1968) A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions of Systems Science and Cybernetics, SSC-4(2), 100-107



IA - Alejandro J. García

Método de búsqueda A*

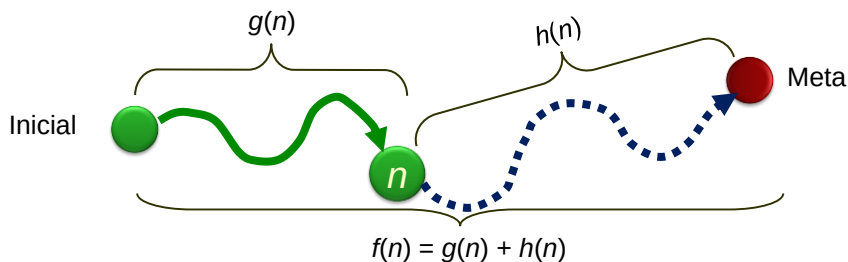


Estrategia de A*: elegir de la frontera el nodo n con menor $f(n) = g(n) + h(n)$

El método A* combina “lowest-cost-first” con “best-first”.

seleccionar(Nodo, [Nodo | RestoLista] , RestoLista).

agregar(ViejaF, Vecinos, NuevaF) :- concatenar (Vecinos, ViejaF, Aux),
ordenar_por_f(Aux, NuevaF).



Hart P. E, Nilsson H. J. and Raphael B. (1968) A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions of Systems Science and Cybernetics, SSC-4(2), 100-107

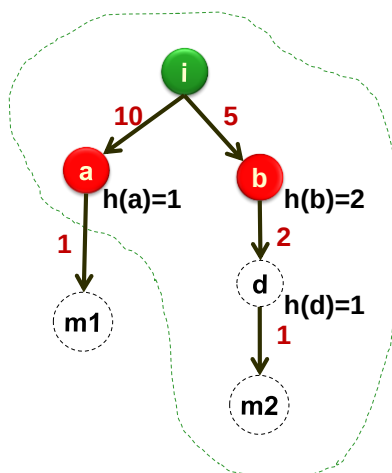
IA - Alejandro J. García



Ejemplo con información heurística



Estrategia de A*: elegir de la frontera el nodo n con menor $f(n) = g(n) + h(n)$



- En este ejemplo, el grafo tiene además costos en los arcos.
- A* considera tanto la información real $g(n)$ como la estimación $h(n)$.

$$f(n) = g(n) + h(n)$$

$$f(a) = 10 + 1 = 11$$

$$f(b) = 5 + 2 = 7$$

$$f(d) = 7 + 1 = 8$$

$$f(m2) = 8 + 0$$

- De esta manera encuentra la meta $m2$.



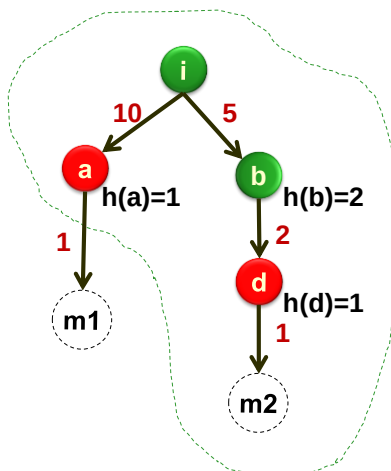
IA - Alejandro J. García



Ejemplo con información heurística



Estrategia de A*: elegir de la frontera el nodo n con menor $f(n) = g(n) + h(n)$



- En este ejemplo, el grafo tiene además costos en los arcos.
- A* considera tanto la información real $g(n)$ como la estimación $h(n)$.

$$f(n) = g(n) + h(n)$$

$$f(a) = 10 + 1 = 11$$

$$f(b) = 5 + 2 = 7$$

$$f(d) = 7 + 1 = 8$$

$$f(m2) = 8 + 0$$

- De esta manera encuentra la meta $m2$.

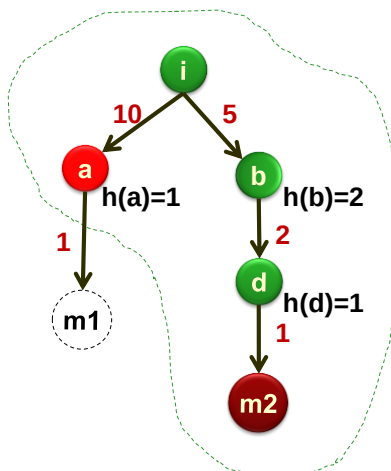


IA - Alejandro J. García

Ejemplo con información heurística



Estrategia de A*: elegir de la frontera el nodo n con menor $f(n) = g(n) + h(n)$



- En este ejemplo, el grafo tiene además costos en los arcos.
- A* considera tanto la información real $g(n)$ como la estimación $h(n)$.

$$f(n) = g(n) + h(n)$$

$$f(a) = 10 + 1 = 11$$

$$f(b) = 5 + 2 = 7$$

$$f(d) = 7 + 1 = 8$$

$$f(m2) = 8 + 0$$

- De esta manera encuentra la meta $m2$.



IA - Alejandro J. García

Función heurística



Hay un **compromiso** importante entre la cantidad de **tiempo** dedicada en computar $h(N)$ y su **exactitud** en la respuesta.

La mejor $h(N)$ posible sería una que dice "exactamente" cuanto falta. Sin embargo, no siempre es posible tenerla, o puede requerir un tiempo de computo prohibitivo.

Como $h(N)$ es una estimación, siempre podrá existir una diferencia con el costo real desde N a la meta.

$h(N)$ se dice que es una **subestimación (underestimate)**, si el valor de $h(N)$ es menor o igual al costo real, del camino más corto desde N hasta una meta alcanzable desde N .

Por ejemplo, $h(N) = 0$ es una subestimación (muy mala por cierto...)



IA - Alejandro J. García

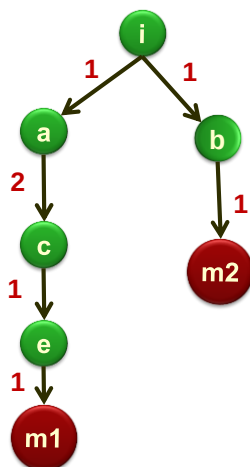
Ejemplos con diferentes $h(n)$



$h_{\text{muybuena}}(N)$
 $h(a)=3$
 $h(b)=1$
 $h(c)=2$
 $h(e)=1$

$h_{\text{buena}}(N)$
 $h(a)=1$
 $h(b)=1$
 $h(c)=2$
 $h(e)=1$

$h_{\text{neutra}}(N)$
 $h(a)=0$ $h(b)=0$
 $h(c)=0$ $h(e)=0$



$h_{\text{mala}}(N)$
 $h(a)=3$
 $h(b)=5$
 $h(c)=2$
 $h(e)=1$

$f(N) = g(N) + h_{\text{mala}}(N)$
 $f(a)=1+3=4$
 $f(b)=1+5=6$
 $f(c)=3+2=5$
 $f(e)=4+1=5$

Realice una traza, e indique cual meta es encontrada con A* para cada heurística.
 ¿En que caso encuentra m1? ¿y m2? ¿por qué encuentra m2 en un caso?



IA - Alejandro J. García

Admisibilidad del Algoritmo A*

La propiedad de encontrar siempre la solución óptima, si es que existe una solución, se denomina **admisibilidad**. (A* la cumple bajo ciertas condiciones). Esto significa que aún en el caso de que el espacio de búsqueda sea infinito, si la solución existe, se encontrará la solución óptima.

Proposición (La admisibilidad de A*):

Si la solución existe, y se cumplen las siguientes tres condiciones, la primera solución encontrada por A* será la óptima:

1. el factor de ramificación es finito (es decir, cada nodo tiene una cantidad finita de vecinos)
2. El costo de los arcos es estrictamente positivo y
3. $h(n)$ es admisible ⁽¹⁾.

Demostración : Ver [PMG] pág. 136-137.



⁽¹⁾ $h(n)$ es admisible si nunca sobrestima el costo de llegar de n a una meta. Esto es, para todo nodo n , $h(n)$ es una cota inferior del mínimo costo de n a un nodo meta.

Dicho de otra manera, las heurísticas admisibles subestiman el camino menos costoso a una meta porque estiman un costo menor al de la realidad.

En las 8 tejas, cantidad de fichas desacomodadas, o la distancia taxicab, son heurísticas admisibles porque su resultado es siempre menor a la realidad, y no es difícil probarlo matemáticamente.

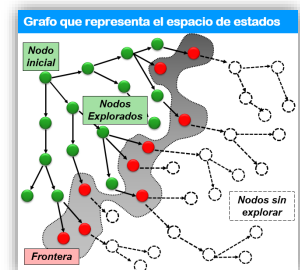
IA - Alejandro J. García

Conclusiones hasta acá...

Métodos de búsqueda informados ("heurísticos") que analizamos:

- El mejor primero (Best-first search)
- Profundidad heurístico (Heuristic Depth-first search)
- A*

En los métodos de búsqueda informada se **utiliza una función heurística aplicada a la información presente en los estados** para realizar una **estimación** de cual nodo en la frontera pueda ser **más promisorio** para llegar más rápidamente a una meta.



Los **tres métodos** que vimos utilizan **diferentes estrategias** de búsqueda, sin embargo, A* es el único de los tres que bajo ciertas condiciones garantiza que aún en el caso que el espacio de búsqueda sea infinito, **si existe una solución óptima, A* la encontrará primero**.

Una característica que comparten estos tres métodos con los métodos de búsqueda ciega que estudiamos antes, es que **en todos los casos los métodos almacenan la frontera de búsqueda**.

Cuando el espacio de búsqueda es muy grande, **almacenar la frontera puede no ser una opción**.

IA - Alejandro J. García

Búsqueda informada en espacios muy grandes

¿Métodos sin frontera?



Métodos de búsqueda **no informados** ("ciegos")

- ✓ búsqueda en profundidad (DFS)
- ✓ búsqueda a la ancho (BFS)
- ✓ Low cost-first (LCFS)

Mantienen una frontera
y luego **eligen a ciegas**
un nodo de la frontera.

Métodos de búsqueda **informados** ("heurísticos")

- ✓ El mejor primero (Best-FS)
- ✓ en profundidad heurístico (HDFS)
- ✓ A*

Mantienen una frontera y
luego **eligen en base a**
una heurística el nodo más
promisorio de la frontera.



Todos esos métodos **almacenan la frontera** para poder realizar una búsqueda sistemática y no saltarse ningún nodo, sin embargo, si el espacio de búsqueda es muy grande, almacenar la frontera puede ser prohibitivo, y además, completamente innecesario...

Propuesta extrema: ¿y si no almacenamos la frontera? ¿Qué decís?



¿Métodos sin frontera?



Para aquellos problemas donde lo que importa es conseguir una meta (estado que satisface el test de meta) pero **no es relevante el camino a la meta** se pueden usar otros métodos diferentes.

Preguntas que voy a intentar responder rápidamente:

¿Cómo buscar en un espacio muy grande sin almacenar la frontera?

¿Cuál podría ser un problema con un espacio de búsqueda muy grande?

¿Cómo podría ser un problema donde no importe el camino a la solución?

¿Cómo puedo definir un nodo meta que no conozco?

Para responder a las últimas tres preguntas voy a utilizar un relato excelente de Adrián Paenza, que cuenta la historia de cómo se resolvió un problema concreto.



IA - Alejandro J. García

“Fixture (a la Dubuc)” [A. Paenza]



El siguiente relato es parte del libro de Adrián Paenza
MATEMÁTICA ... ¿ESTÁS AHÍ? Episodio 2

“... No sé si habrá prestado atención alguna vez a un fixture de fútbol, la programación de todos los partidos que se juegan en el año. Un fixture estándar consiste en 19 fechas en las que los 20 equipos jueguen todos contra todos⁽¹⁾.

Además, semana tras semana los equipos alternan su condición de local y visitante.

⁽¹⁾ *Eso fue hasta 2014, pero luego en 2015....* <http://www.afa.org.ar/>



Mil gracias Adrián por tu relato. Es excelente.

Observación: como tuve que resumirlo para adaptarlo al formato de esta clase, te recomiendo leer la versión completa a partir de la pág. 194 del libro original, que está en línea en: <http://cms.dm.uba.ar/material/paenza/libro2/>
Además, te recomiendo leer todo el libro que está genial.



2006, Siglo XXI Editores
ISBN-10: 987-1220-64-2



IA - Alejandro J. García

“Fixture” [A. Paenza]

Confeccionar un fixture no debería ser una tarea difícil. Sin embargo, lo invito a que lo intente para comprobar el grado de dificultad que presenta.

Si hubiera 4 equipos, habría 3 fechas (que son 3! permutaciones), y en cada fecha hay 4 formas de organizar local y visitante, lo que lleva a $4.3! = 4!$ fixtures posibles para 3 equipos.

Con 20 equipos hay 19 fechas y $20! = 2.432.902.008.176.640.000$ fixtures distintos. Casi dos trillones y medio de fixtures.

Cuando los partidos se jugaban todos el día domingo (sí, aunque parezca mentira ...), todo era relativamente sencillo. Pero la televisión cambió todo...

La TV creó ciertas restricciones: seleccionar un partido para los viernes con un equipo de los denominados “grandes” [...]. Después se agregó uno para los sábados, con una transmisión originada en el interior del país [...] Luego se sumó un partido los lunes entre dos clubes “chicos”.



IA - Alejandro J. García

Buscando un “Fixture”

Un fixture es la programación de todos los partidos de un campeonato. La TV creó ciertas restricciones: seleccionar un partido para los viernes con un equipo de los denominados “grandes” [...]. Después se agregó uno para los sábados, con una transmisión originada en el interior del país [...] Luego se sumó un partido los lunes entre dos clubes “chicos”.

Una fecha X (10 partidos)	
Local	Visita
VIERNES	
SÁBADO	
DOMINGO	
...	...
LUNES	

Cada fixture es un nodo en el espacio de búsqueda.

Un cambio en una fecha lleva de un nodo a otro.

La cantidad de sucesores de un nodo es enorme.

El espacio de búsqueda es enorme.

Hay 1.055.947.052.160.000 fixtures distintos.

Un fixture es “TV-compatible” si cumple todas las restricciones dichas antes en el relato....

Observación impotante: En este problema importa encontrar un fixture que sea TV-compatible (meta), pero el camino del nodo inicial a la meta no es relevante.

Un fixture con 19 fechas				
Fecha 1	Fecha 2	Fecha 3	Fecha 4	Fecha
Fecha	Fecha	Fecha	Fecha	Fecha
Fecha X	Fecha	Fecha	Fecha	Fecha
Fecha	Fecha	Fecha	Fecha 19	



IA - Alejandro J. García

“Fixture” [A. Paenza]

Si uno intenta hacerlo a mano (y créame que hubo mucha gente que se lo propuso) son tantos los ajustes que hay que hacerle a un fixture para que cumpla con todas esas restricciones [impuestas por la TV], que ya se dudaba de que un fixture así existiera, o que fuera posible armarlo. ¿Qué hacer?

Una vez [aplicadas ciertas restricciones], el número total de fixtures diferentes [para 20 equipos] es de 1.055.947.052.160.000.

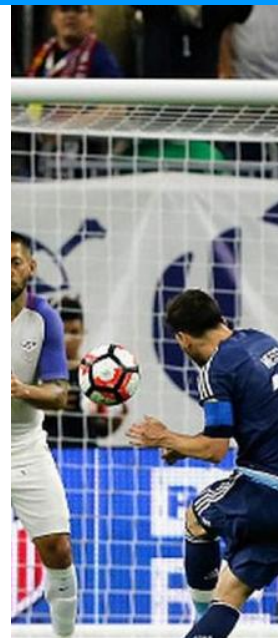
Surgía inmediatamente una pregunta: ¿quién los revisaría para saber cuál o cuáles eran los que servían? ⁽¹⁾

Y un tema clave, muy importante: ¿cuánto tiempo se tardaría en examinarlos a todos?

Por ejemplo: a razón de investigar 5.000 fixtures por segundo llevaría...

más de 6.000 años [con 20 equipos].

⁽¹⁾ Parece un problema de búsqueda ¿no? “dentro del enorme espacio de búsqueda de fixtures, encontrar un fixture que cumpla todas las restricciones impuestas por la TV (o que sea TV-compatible)”



IA - Alejandro J. García

“Fixture” [A. Paenza]

En ese momento, enero de 1995, me derivaron el problema para ver si algún matemático (como yo sostenía) era capaz de resolverlo, contemplando todas las restricciones señaladas.

Le di el problema al doctor [Eduardo Dubuc](#), profesor titular del Departamento de Matemáticas [UBA] desde hace años, y uno de los más prestigiosos que tiene el país. ...

Me formuló las preguntas lógicas para alguien que sigue el fútbol sólo como aficionado. Cerró la carpeta que contenía los datos, se sacó los anteojos que usa siempre, me miró en silencio durante un rato y me preguntó:

—¿Vos estás seguro de que este problema tiene solución?

—No sé, pero seguro que si la tiene, vos sos la persona para encontrarla.

Unos días más tarde, me entregó un fixture junto con algunos comentarios escritos. Recuerdo uno en particular: “El problema está resuelto de la mejor manera posible”.



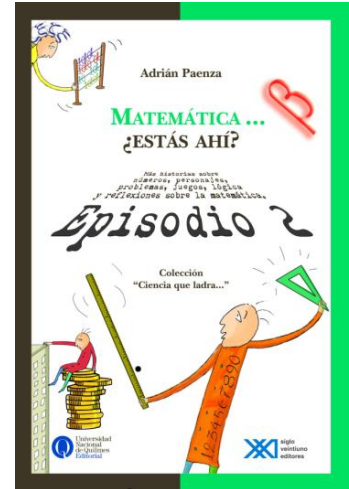
IA - Alejandro J. García

“Fixture (a la Dubuc)” [A. Paenza]

Para resolver el problema, Dubuc utilizó un programa con un algoritmo de búsqueda que usa “Recocido Simulado” ⁽¹⁾.

Eligiendo al azar dónde empezar (es decir, se elige un fixture estándar cualquiera para comenzar), después de revisar entre 500.000 y un millón de fixtures en unos 20 minutos y utilizando una PC 386 de aquella época, el programa que diseñó Eduardo Dubuc encontraba un fixture que resolvía el problema.

La Asociación de Fútbol Argentino (AFA) implementó su uso a partir del campeonato Apertura de 1995.



⁽¹⁾ “Recocido Simulado”, es una versión estocástica del método de búsqueda “el escalador” (en inglés Hill Climbing).

IA - Alejandro J. García

“Fixture a la Dubuc”

Dado un fixture cualquiera, a cada condición que no cumple para ser “TV-compatible” se le asocia un número y la suma de todos esos números representa la “multa” del fixture.

Un FIXTURE con 19 fechas				
Fecha 1	Fecha 2	Fecha 3	Fecha 4	Fecha
Fecha	Fecha	Fecha	Fecha	Fecha
Fecha	Fecha	Fecha	Fecha	Fecha
Fecha	Fecha	Fecha	Fecha 19	

Cada fixture es un nodo de un espacio de búsqueda. Usando la multa como heurística se puede utilizar HC con recocido simulado para buscar un nodo con la mínima multa.

MULTA = 86

Si la multa es 0 (cero) entonces tengo una solución.

IA - Alejandro J. García

Método del Escalador

Hill Climbing (HC)

Hill Climbing (Escalador o Trepada)



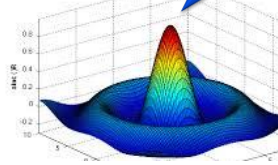
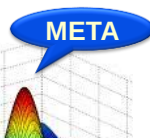
El método del escalador es un método de búsqueda diseñado para aplicarse a dominios donde se tiene una **función heurística**.

Está pensado para encontrar una meta rápido, en un espacio de búsqueda muy grande donde almacenar la frontera es prohibitivo.

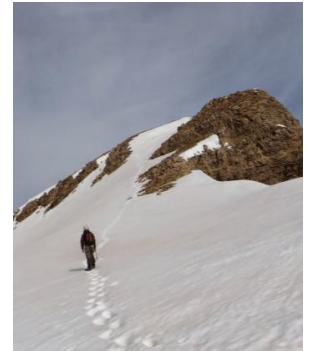
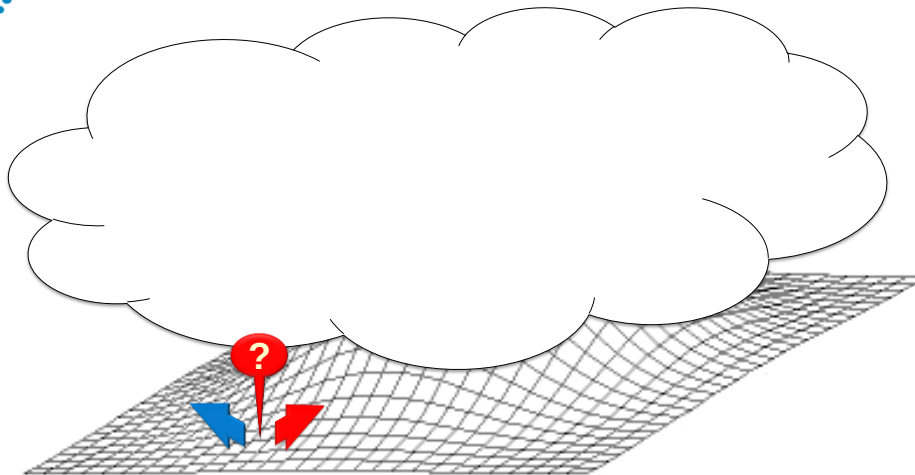
- La **solución** de este método **es un nodo meta**.
- No almacena, ni retorna el camino recorrido desde el nodo inicial al nodo meta.
- Tampoco almacena la frontera de búsqueda.
- La forma de orientar la búsqueda es **en la "dirección" donde $h(N)$ es mínima (o máxima)**.



También **se aplica a problemas de optimización** donde se busca encontrar **el máximo de una función**.



Hill Climbing (Escalador o Trepada)

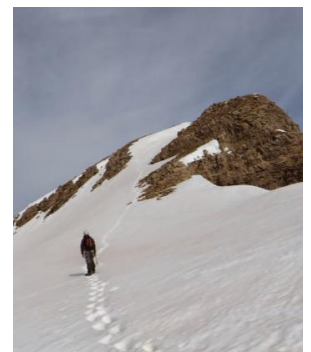
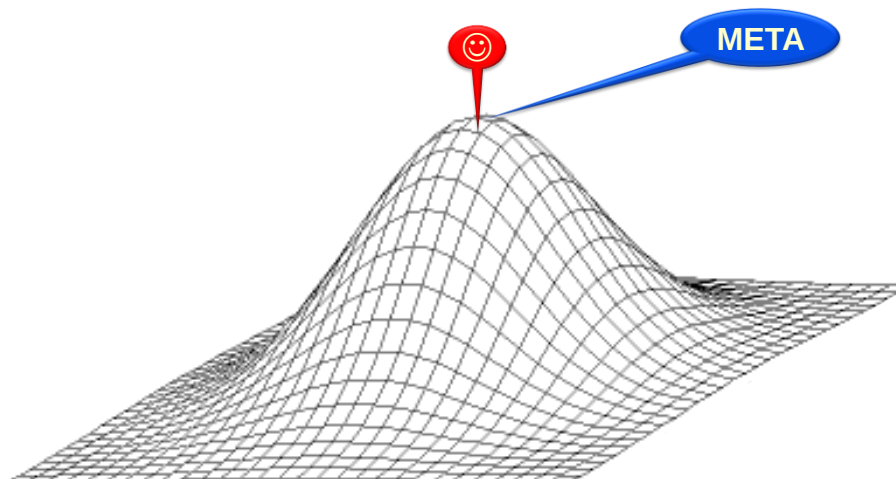


Si quisieras llegar a la cima de una colina y no podés ver todo el terreno, solo podés ver la zona vecina próxima ¿irías en la dirección de la flecha azul o la roja?



IA - Alejandro J. García

Hill Climbing (Escalador o Trepada)



En la mayoría de las colinas, seguir la mayor pendiente te lleva inevitablemente a la cima.



IA - Alejandro J. García

Hill Climbing (HC)



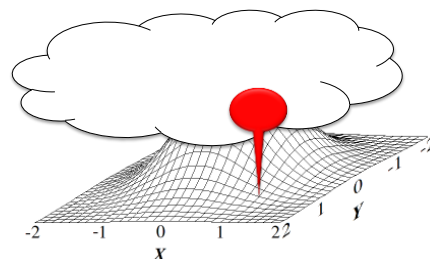
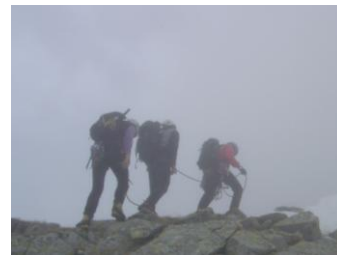
Algoritmo HC:

Inicialmente selecciona un nodo N para comenzar la búsqueda (esto puede ser al azar) y luego mantiene un solo nodo en cada etapa.

Para continuar la búsqueda, de todos los sucesores de N que sean "*mejores*" que N elige el mejor (si es que existe, y si hubiera más de uno posible para elegir, elige uno al azar).

Se detiene si no hay ningún sucesor *mejor* que N .

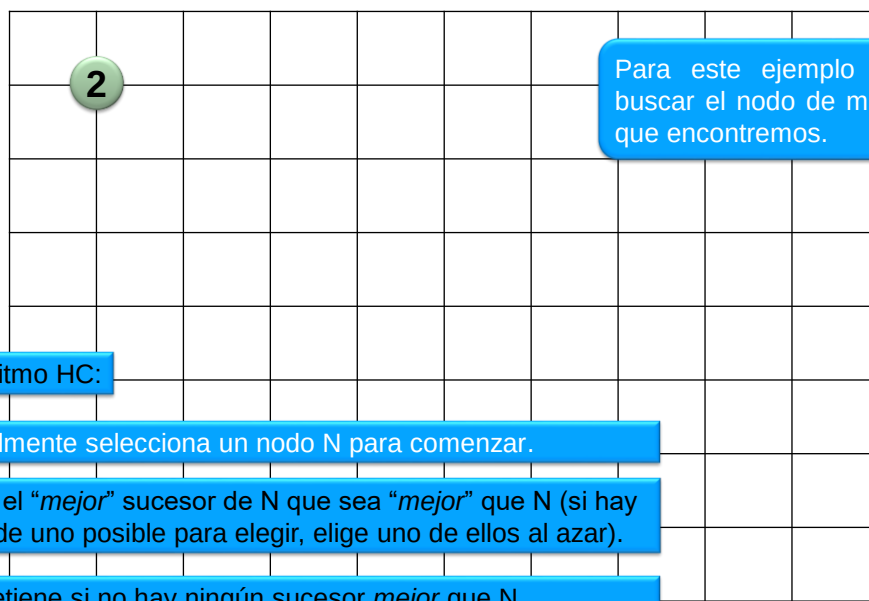
La definición de "*mejor*" sucesor dependerá del tipo de problema que se está resolviendo: podría ser el "*menor valor de $h(x)$* " si se usa una función heurística; o también podría ser "*mayor que N* " si se está buscando el máximo de una función.



IA - Alejandro J. García



Ejemplo: nodo inicial



Para este ejemplo vamos a buscar el nodo de mayor valor que encontremos.

Algoritmo HC:

Inicialmente selecciona un nodo N para comenzar.

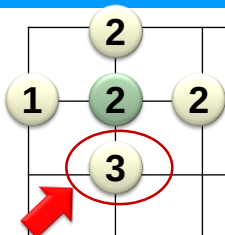
Elige el "*mejor*" sucesor de N que sea "*mejor*" que N (si hay más de uno posible para elegir, elige uno de ellos al azar).

Se detiene si no hay ningún sucesor *mejor* que N .

IA - Alejandro J. García



Ejemplo: nodo inicial y sus sucesores (vecinos)



Para este ejemplo usaremos "mejor" como "mayor que", pero el mismo ejemplo podría reformularse con valores de una función heurística y "mejor" sería "menor que".

Algoritmo HC:

Inicialmente selecciona un nodo N para comenzar.



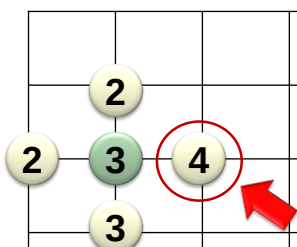
Elige el "mejor" sucesor de N que sea "mejor" que N (si hay más de uno posible para elegir, elige uno de ellos al azar).

Se detiene si no hay ningún sucesor *mejor* que N.



IA - Alejandro J. García

Primera elección y sus vecinos



¿Cuánta memoria necesita?

Valores en memoria

Nodo N	3
Mejor vecino encontrado	4

Algoritmo HC:

Inicialmente selecciona un nodo N para comenzar.



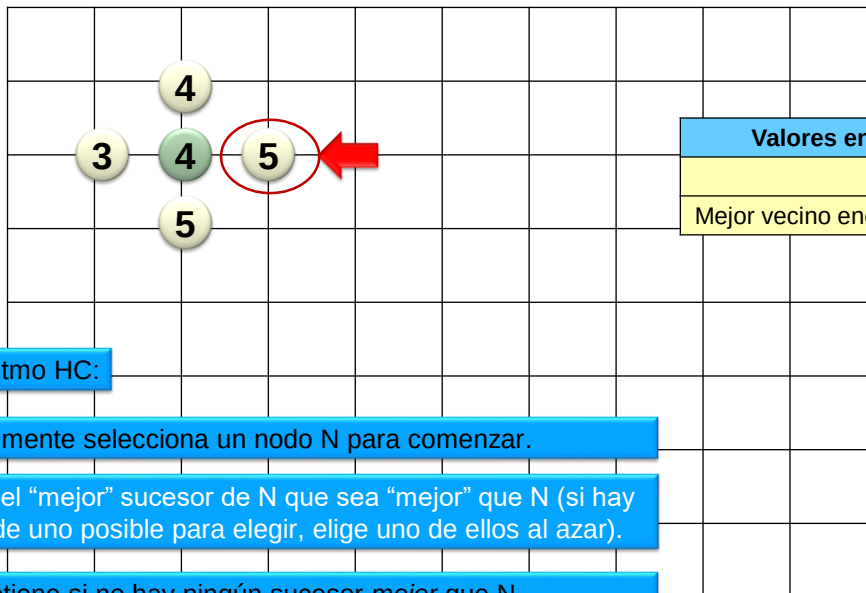
Elige el "mejor" sucesor de N que sea "mejor" que N (si hay más de uno posible para elegir, elige uno de ellos al azar).

Se detiene si no hay ningún sucesor *mejor* que N.



IA - Alejandro J. García

Si hay varios candidatos elige uno al azar



Algoritmo HC:

Inicialmente selecciona un nodo N para comenzar.



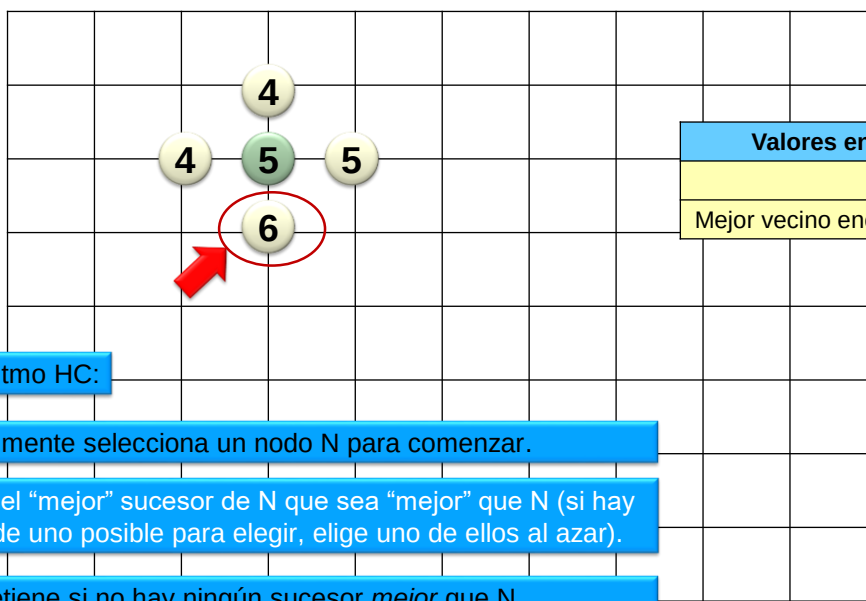
Elige el "mejor" sucesor de N que sea "mejor" que N (si hay más de uno posible para elegir, elige uno de ellos al azar).

Se detiene si no hay ningún sucesor *mejor* que N.



IA - Alejandro J. García

Tercera elección y sus vecinos



Algoritmo HC:

Inicialmente selecciona un nodo N para comenzar.



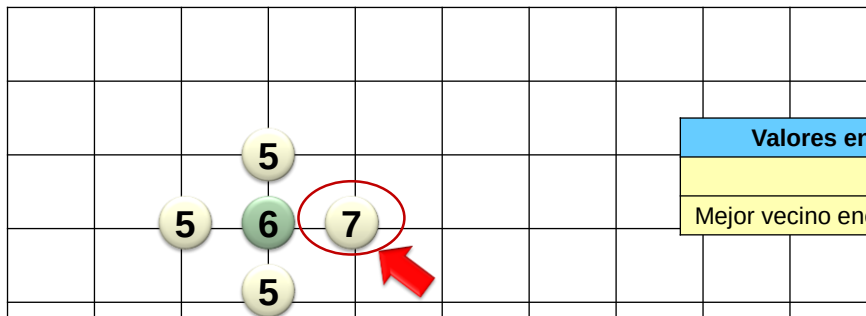
Elige el "mejor" sucesor de N que sea "mejor" que N (si hay más de uno posible para elegir, elige uno de ellos al azar).

Se detiene si no hay ningún sucesor *mejor* que N.



IA - Alejandro J. García

Cuarta elección y sus vecinos



Valores en memoria

Nodo N	6
Mejor vecino encontrado	7

Algoritmo HC:

Inicialmente selecciona un nodo N para comenzar.



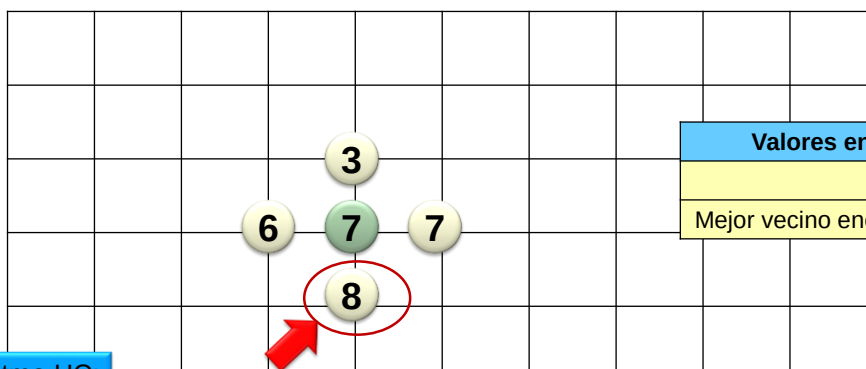
Elige el "mejor" sucesor de N que sea "mejor" que N (si hay más de uno posible para elegir, elige uno de ellos al azar).

Se detiene si no hay ningún sucesor *mejor* que N.



IA - Alejandro J. García

Quinta elección y sus vecinos



Valores en memoria

Nodo N	7
Mejor vecino encontrado	8

Algoritmo HC:

Inicialmente selecciona un nodo N para comenzar.



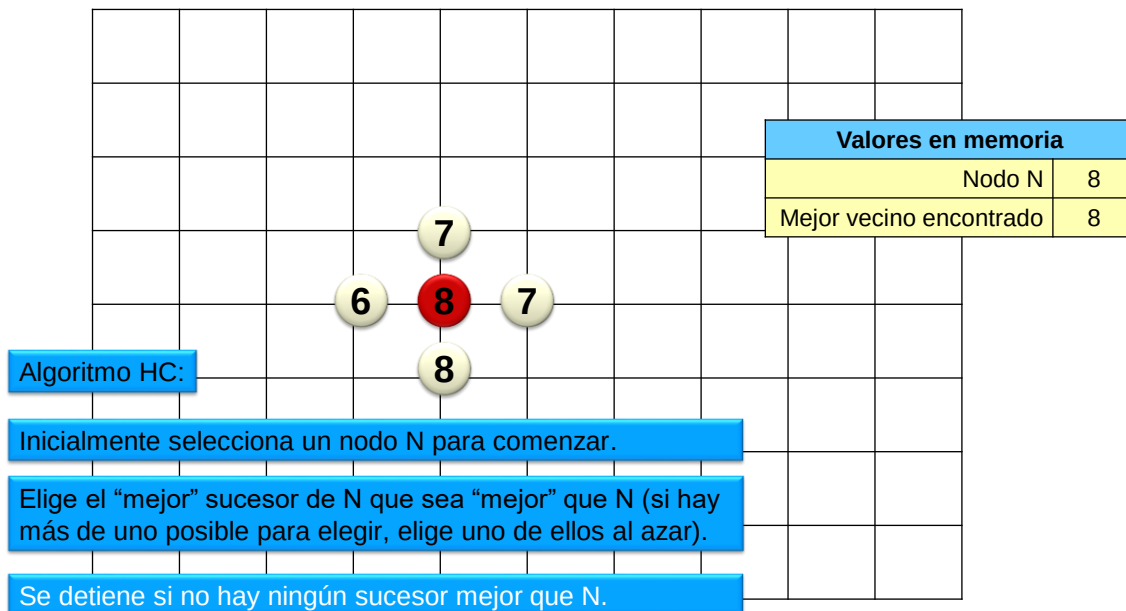
Elige el "mejor" sucesor de N que sea "mejor" que N (si hay más de uno posible para elegir, elige uno de ellos al azar).

Se detiene si no hay ningún sucesor *mejor* que N.



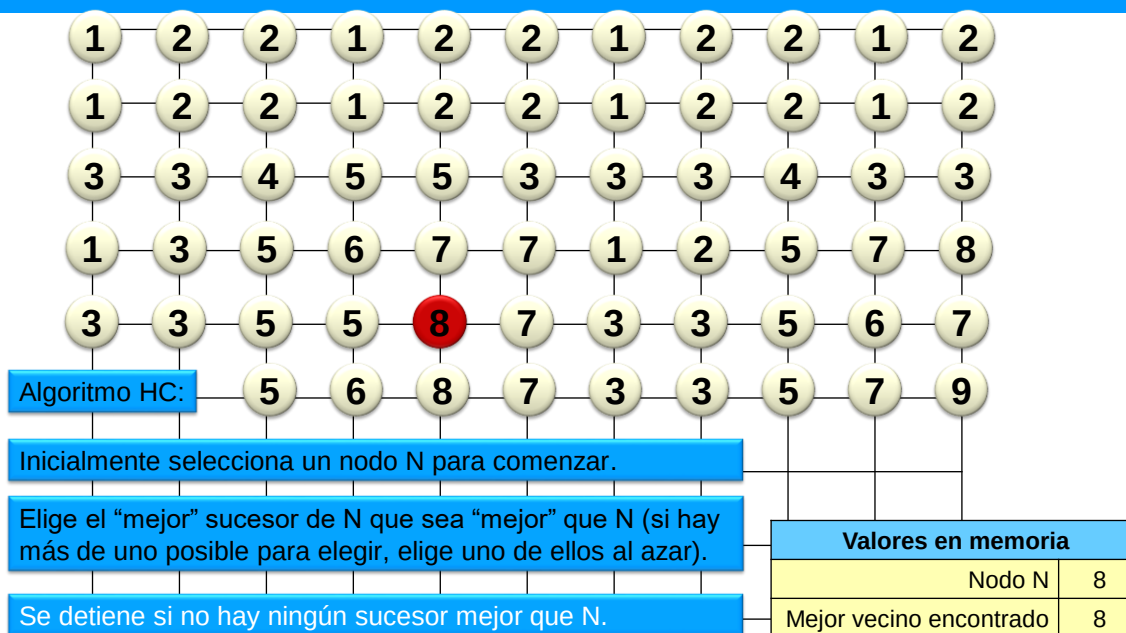
IA - Alejandro J. García

Sexta elección y encuentra la meta porque no hay ninguno mejor



IA - Alejandro J. García

La meta encontrada



IA - Alejandro J. García

Hill Climbing (Escalador o Trepada)



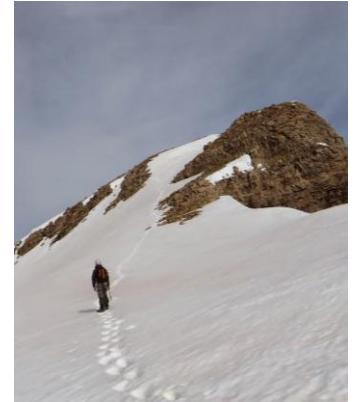
HC está pensado para encontrar una solución rápido.

También es llamado **greedy local search**.

Mantener toda la frontera puede garantizar ser completo pero puede ser prohibitivo en algunos dominios.

HC solo mantiene un nodo, pero **no garantiza ser completo**.

A continuación veremos algunos **problemas con los que se puede encontrar HC**, y luego veremos como solucionarlos.



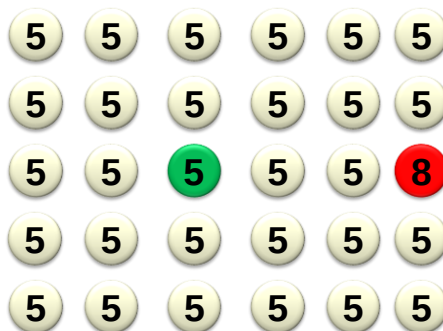
IA - Alejandro J. García

Problemas en HC: (1) mesetas



Se detiene si no hay ningún sucesor mejor que N.

Si en el espacio de búsqueda se encuentra con una “**meseta**” (plateaux) donde no hay ningún vecino mejor, entonces se detiene.

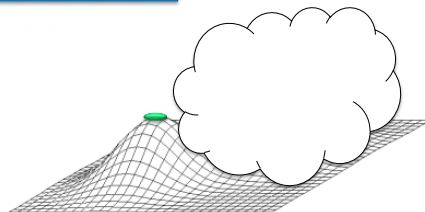
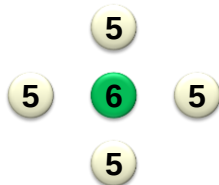


IA - Alejandro J. García

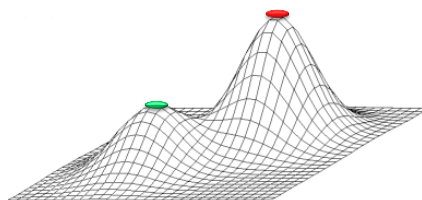
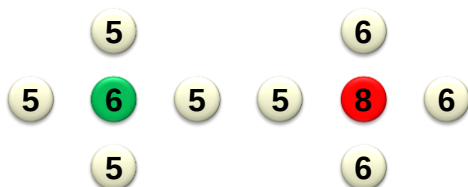
Problemas en HC: (2) máximos locales



Se detiene si no hay ningún sucesor mejor que N.



Se detiene si en el espacio de búsqueda se encuentra con un máximo local (no hay ningún vecino mejor).



Aunque podría ocurrir que hay un máximo global al cual no podrá llegar...

IA - Alejandro J. García

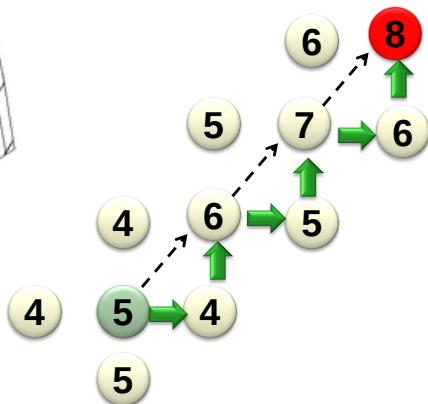
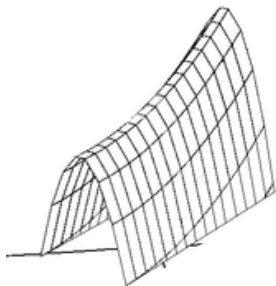
Problemas en HC: (3) Crestas



Se detiene si no hay ningún sucesor mejor que N.

“Crestas” (ridges): aunque hay un camino ascendente (flechas punteadas), no puede avanzar por ahí porque no son vecinos.

Tampoco puede usar el camino de las flechas solidas porque no son mejores.



Puede pensarse también como una “secuencia de máximos locales que va ascendiendo” y por la cual no se puede avanzar en forma directa.



IA - Alejandro J. García

Random walk vs HC

El método **HC** utiliza una **heurística** que dirige la búsqueda, y eso le permite ir más directo a una meta.

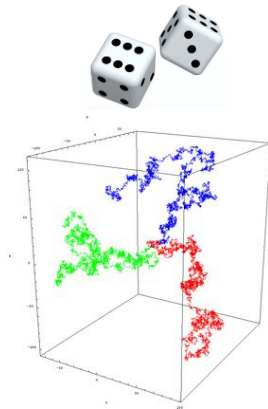
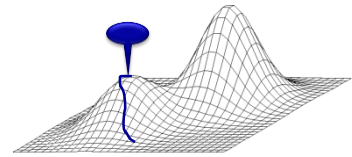
Sin embargo, por la forma en que está definido, **puede quedar "detenido"** en lugares que no son la solución buscada.

Random Walk (RW) o caminar aleatoriamente **elige al azar** un sucesor hasta llegar a una meta.

RW no usa ninguna heurística y puede tomar cualquier camino de manera aleatoria (a ciegas) hasta "toparse" con una solución.

El método **RW** puede tener ventajas en esas situaciones donde **HC** se queda detenido.

Por lo tanto, una idea razonable es **combinar HC y RW de la mejor manera posible**.



IA - Alejandro J. García

Variantes de Hill Climbing con aleatoriedad

Para solucionar los problemas de **HC** existen variantes que incluyen aleatoriedad:

Random-restart hill climbing: lleva a cabo una serie de corridas de **HC** comenzando de diferentes nodos iniciales generados aleatoriamente.

First-choice hill climbing: cuando hay muchos vecinos de un nodo **N** (por ejemplo 1000) se puede elegir al azar hasta que encuentra uno que sea mejor que **N**, en lugar de elegir siempre el mejor de todos.

Simulated Annealing (Recocido Simulado): es una variante de **HC** en la cual al comienzo se eligen más veces (al azar) vecinos que no mejoren al nodo; y luego al ir avanzando ("enfriamiento") se eligen menos a aquellos vecinos que no mejoran.

Hill climbing en paralelo: otra alternativa para reducir la posibilidad de quedar "estancado" es llevar en paralelo diferentes corridas de **HC** comenzando de diferentes nodos iniciales.



IA - Alejandro J. García

Hill Climbing con “recocido simulado” (simulated annealing)

Hill Climbing (HC) con “recocido simulado”



En metalurgia, **recocido** (en inglés **annealing**) es un proceso usado para endurecer o fortalecer un metal (o vidrio) mediante su calentamiento, y posterior enfriamiento gradual (logrando que el material llegue un estado cristalino de baja energía).

La forma de **combinar HC y RW**, que se inspira en el proceso de recocido es llamada **recocido simulado** o “simulated annealing” (**SA**)

SA es una variante de HC que sigue el siguiente esquema:

Al comienzo de la ejecución (“alta temperatura”) elegir más veces (al azar) vecinos que no mejoren al nodo; y luego al ir avanzando (“enfriamiento”) elegir menos a aquellos vecinos que no mejoran.

También se lo llama **stochastic HC**

(Estocástico: que depende del azar / involucra probabilidad)



“Fixture a la Dubuc” [A. Paenza]

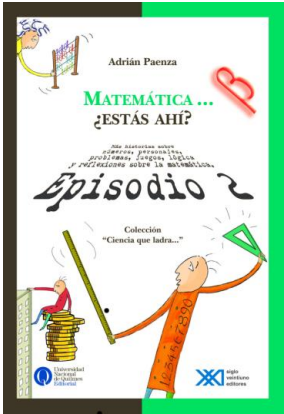
Parte final del relato de Adrián Paenza. Disfrutemos juntos.

“...Para resolver el problema Dubuc utilizó Recocido Simulado.

No es posible que incluya aquí las precisiones sobre el método del recocido simulado en sí mismo, pero, en todo caso, vale la pena decir que involucra movimientos al azar, la teoría de probabilidades y se inspira en un análisis probabilístico de lo que sucede cuando se enfría lentamente el vidrio en la fabricación de botellas (de ahí el nombre recocido), y es simulado, porque se usa una computadora.

Primero hay que empezar por calificar los fixtures. ¿Qué quiere decir esto? Elijan un fixture estándar cualquiera [al azar]. Lo más probable es que no cumpla la mayoría de los requisitos que se necesitan.

Entonces, a Eduardo se le ocurrió poner una multa por cada restricción que no cumpliera.



IA - Alejandro J. García

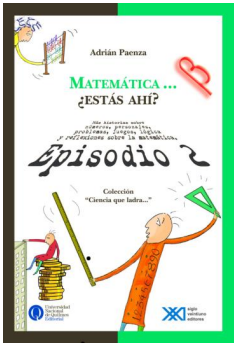
“Fixture a la Dubuc” [A. Paenza]

Entonces, a Eduardo se le ocurrió poner una multa por cada restricción que no cumpliera.

Por ejemplo, si en el fixture que había elegido, en la primera fecha no había partido para los viernes, le ponía tres puntos de multa. Si le faltaba partido desde el interior, dos puntos de penalidad. Y así siguió hasta agotar la primera fecha. Pasó entonces a la segunda, y esencialmente las recorrió todas acumulando las multas que sufrían en el camino.

Al finalizar el proceso, ese fixture tenía adosada una cantidad de puntos en contra, es decir, una multa.

En definitiva, cuanto mayor fuera la multa de un fixture, peor era. Como se advierte, el objetivo de Eduardo era encontrar el o los fixtures que tuvieran multa cero [minimizar]. Es decir, aquellos programas de partidos que no infringieran ninguna de las normas pedidas. ¿Existirían? ¿Tendría solución el problema?



Un FIXTURE con 19 fechas				
Fecha 1	Fecha 2	Fecha 3	Fecha 4	Fecha
Fecha	Fecha	Fecha	Fecha	Fecha
Fecha	Fecha	Fecha	Fecha	Fecha
Fecha	Fecha	Fecha	Fecha	Fecha 19

MULTA = 86

IA - Alejandro J. García

“Fixture a la Dubuc”

Dado un fixture cualquiera, a cada condición que no cumple para ser “TV-compatible” se le asocia un número y la suma de todos esos números representa la “multa” del fixture.

Un FIXTURE con 19 fechas				
Fecha 1	Fecha 2	Fecha 3	Fecha 4	Fecha
Fecha	Fecha	Fecha	Fecha	Fecha
Fecha	Fecha	Fecha	Fecha	Fecha
Fecha	Fecha	Fecha	Fecha 19	

MULTA = 86

Cada fixture es un nodo de un espacio de búsqueda. Usando la multa como heurística se puede utilizar HC con recocido simulado para buscar un nodo con la mínima multa.

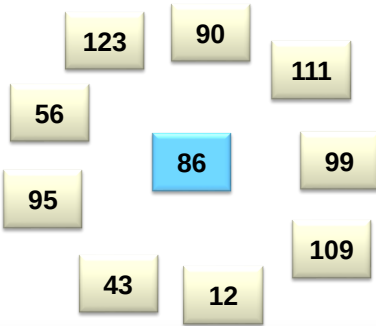
Si la multa es 0 (cero) entonces tengo una solución.

IA - Alejandro J. García



Búsqueda con HC y recocido simulado

Dado un fixture cualquiera, a cada condición que no cumple para ser “TV-compatible” se le asocia un número y la suma de todos esos números representa la “multa” del fixture.



Cada fixture es un nodo de un espacio de búsqueda. Usando la multa como heurística se puede utilizar HC con recocido simulado para buscar un nodo con la mínima multa.

Si la multa es 0 (cero) entonces tengo una solución.

SA es una variante de HC que sigue el siguiente esquema:
Al comienzo de la ejecución (“alta temperatura”) elegir más veces (al azar) vecinos que no mejoren al nodo; y luego al ir avanzando (“enfriamiento”) elegir menos a aquellos vecinos que no mejoran.

IA - Alejandro J. García

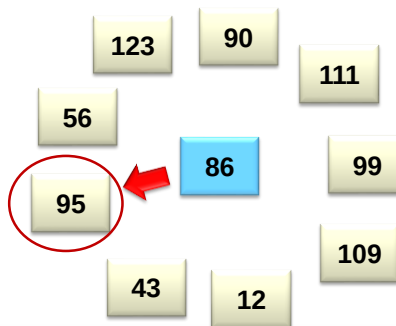


Búsqueda con HC y recocido simulado

Dado un fixture cualquiera, a cada condición que no cumple para ser "TV-compatible" se le asocia un número y la suma de todos esos números representa la "multa" del fixture.

Cada fixture es un nodo de un espacio de búsqueda. Usando la multa como heurística se puede utilizar HC con recocido simulado para buscar un nodo con la mínima multa.

Si la multa es 0 (cero) entonces tengo una solución.



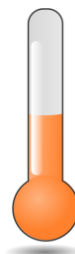
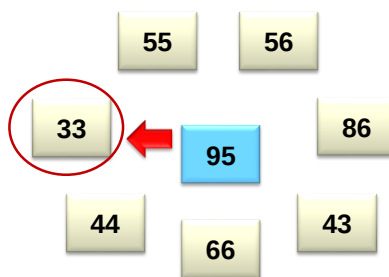
SA es una variante de HC que sigue el siguiente esquema:
Al comienzo de la ejecución ("alta temperatura") elegir más veces (al azar) vecinos que no mejoren al nodo; y luego al ir avanzando ("enfriamiento") elegir menos a aquellos vecinos que no mejoran.



IA - Alejandro J. García

Búsqueda con HC y recocido simulado

Dado un fixture cualquiera, a cada condición que no cumple para ser "TV-compatible" se le asocia un número y la suma de todos esos números representa la "multa" del fixture.



Si la multa es 0 (cero) entonces tengo una solución.

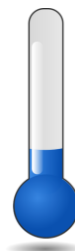
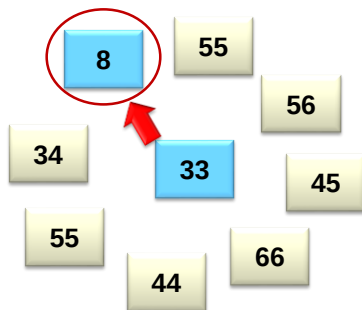
SA es una variante de HC que sigue el siguiente esquema:
Al comienzo de la ejecución ("alta temperatura") elegir más veces (al azar) vecinos que no mejoren al nodo; y luego al ir avanzando ("enfriamiento") elegir menos a aquellos vecinos que no mejoran.



IA - Alejandro J. García

Búsqueda con HC y recocido simulado

Dado un fixture cualquiera, a cada condición que no cumple para ser "TV-compatible" se le asocia un número y la suma de todos esos números representa la "multa" del fixture.



Si la multa es 0 (cero) entonces tengo una solución.

SA es una variante de HC que sigue el siguiente esquema:

Al comienzo de la ejecución ("alta temperatura") elegir más veces (al azar) vecinos que no mejoren al nodo; y luego al ir avanzando ("enfriamiento") elegir menos a aquellos vecinos que no mejoran.

IA - Alejandro J. García

"Fixture a la Dubuc" [A. Paenza]

Una aclaración muy importante: seguro que quienes concibieron, usan o usaron el recocido simulado no tuvieron in mente resolver un problema de estas características.

Pero ahí también reside la capacidad de un [profesional] para saber que hay una herramienta que, en principio, no parece haber sido construida para esta ocasión en particular y, sin embargo, con una adaptación no sólo se transformó en útil, sino que permitió encontrar la solución.



Mil gracias de nuevo Adrián por tu relato. Es excelente.

También mil gracias Eduardo por tu resolución del problema.

Estoy en deuda con ambos, porque creo que es el mejor ejemplo que podría haber encontrado para explicar este tema de búsqueda.



IA - Alejandro J. García

Local Beam Search (LBS)

Con respecto al uso de memoria, en un extremo está mantener siempre almacenada TODA la frontera y en el otro extremo directamente no tener frontera como HC. Se puede hacer algo “intermedio” como en la estrategia llamada **Local Beam Search** que mantiene en memoria una cantidad fija de K estados como “frontera”. **Beam**: (haz) “grupo de búsqueda que no se dispersa”.



- Local Beam Search :
- Comienza con K estados generados al azar.
 - En cada paso se calculan todos los sucesores de los K nodos.
- Si uno de ellos es meta: lo retorna y para, si no: selecciona los mejores K sucesores de la lista completa y repite el proceso.

Pregunta desafiante:
¿**Local Beam Search** es igual a ejecutar K HCs en paralelo?

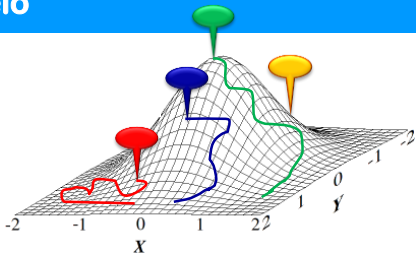


IA - Alejandro J. García

Beam Search vs. HC en paralelo

¿**Local Beam Search** es igual a correr K HCs en paralelo?

¿Cuáles elige cada método?



	Nodo 1			Nodo 2		Nodo 3		Nodo 4			Nodo 5			
sucesores:	5	3	2	6	7	1	2	3	8	5	2	1	2	3
HC paralelo:														
Beam S:														



IA - Alejandro J. García

Beam Search vs. HC en paralelo

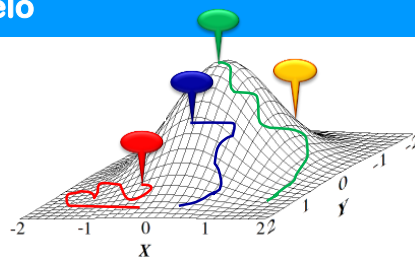


¿Local Beam Search es igual a correr K HCs en paralelo?

Respuesta:

No es lo mismo.

Ya que cada HC elegirá el mejor sucesor sin tener en cuenta a los demás (siempre habrá un vecino de cada uno de los K nodos). Veamos un ejemplo con K = 5.



	Nodo 1			Nodo 2		Nodo 3		Nodo 4			Nodo 5			
sucesores:	5	3	2	6	7	1	2	3	8	5	2	1	2	3
HC paralelo:	5				7		2		8					3
Beam S:	5			6	7				8	5				



También puede aplicarse recocido simulado a LBS, llamado “**stochastic beam search**” (SBS), donde se eligen al azar K nuevos, pero con una probabilidad que depende de su valor.



IA - Alejandro J. García

Referencias



- [PMG] Poole, D., Mackworth, A., and Goebel, R. *Computational Intelligence: A Logical Approach*. (Section 4.7)
- [RN] Russel, S., and Norvig, P. *Artificial Intelligence: A Modern Approach*.
4: Beyond classical search
- [A. Paenza] Adrián Paenza.
MATEMÁTICA ... ¿ESTÁS AHÍ? Episodio 2
(2006) En línea: <http://cms.dm.uba.ar/material/paenza/libro2/>



IA - Alejandro J. García

Video con material complementario



IA - Alejandro J. García

Esto no termina acá...



Gracias por tu atención.

つづく

IA - Alejandro J. García